WHY

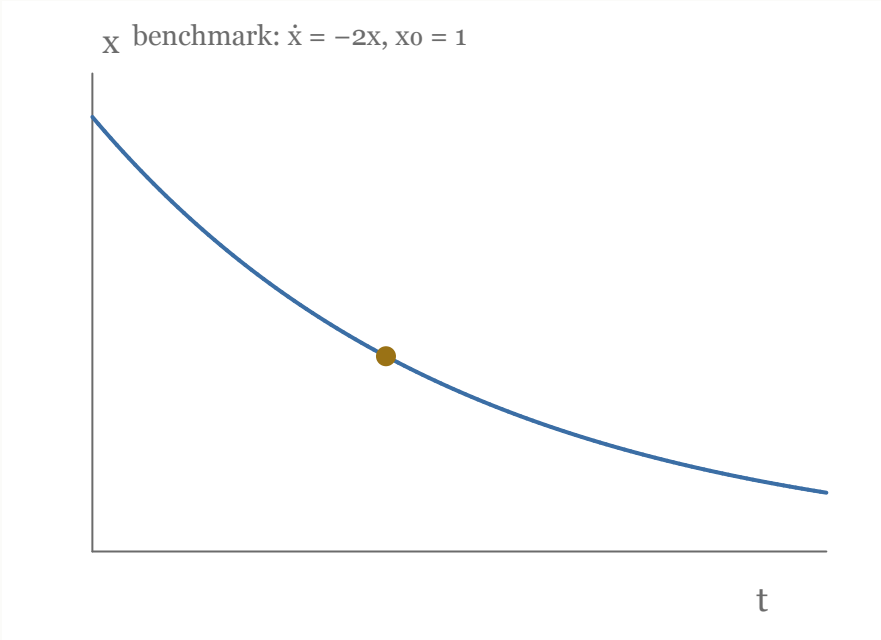
ODE and Simulation

How does a simulator advance the world?

$$\dot{x} = f(x, t), \qquad x(t_0) = x_0$$

- `x` state column
- `f` rate of change
- `t` time; to is the start
- x_0 known initial state

A model gives a velocity. A simulator must produce a future state.



`Input/state in · model out · error res · data dat`

OBJECTS

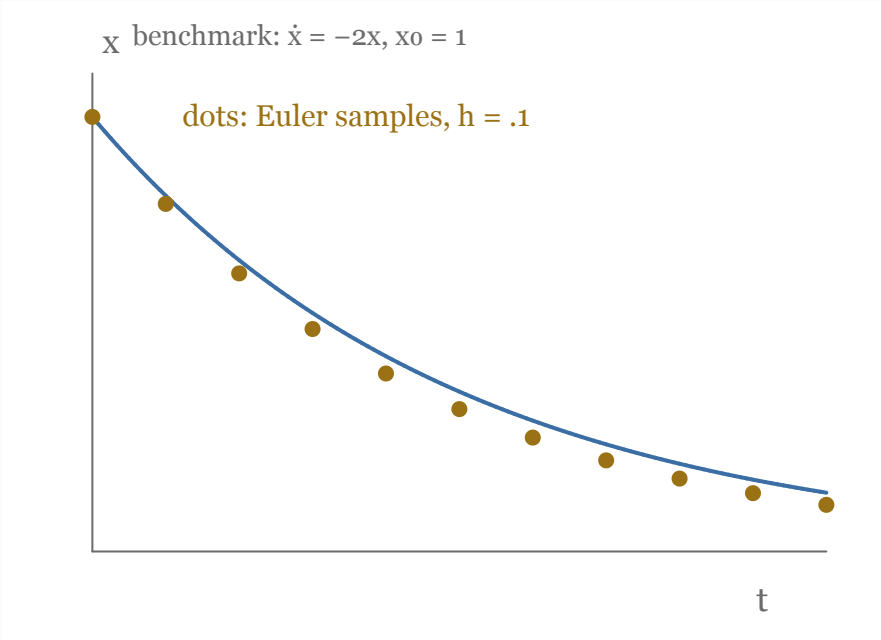
ODE and Simulation

A trajectory and its stored samples

$$t_k = t_0 + kh, \qquad x_k \approx x(t_k)$$

- `h` positive time step
- `k` integer sample index
- `x(t)` continuous exact state
- x_k numerical approximation

The curve belongs to the model; the dots belong to the computation.



DEFINITION

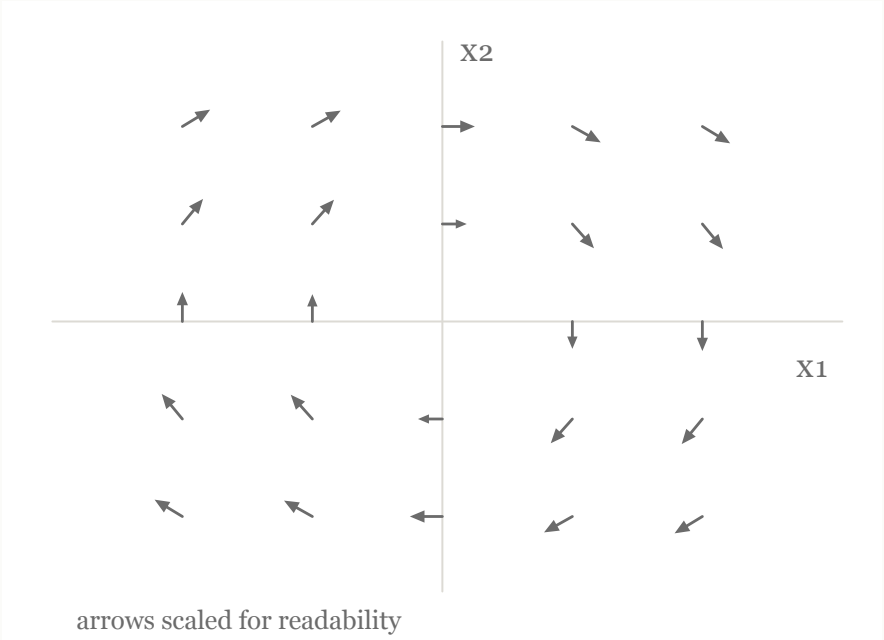
Velocity Fields

An arrow gives a rate, not a destination

$$\dot{x} = f(x, t) \in \mathbb{R}^d$$

- `x` d-component state column
- `f(x,t)` d-component state rate
- `t` time held fixed in a field picture

Arrow direction and magnitude describe instantaneous motion.



arrows scaled for readability

ASSUMPTIONS

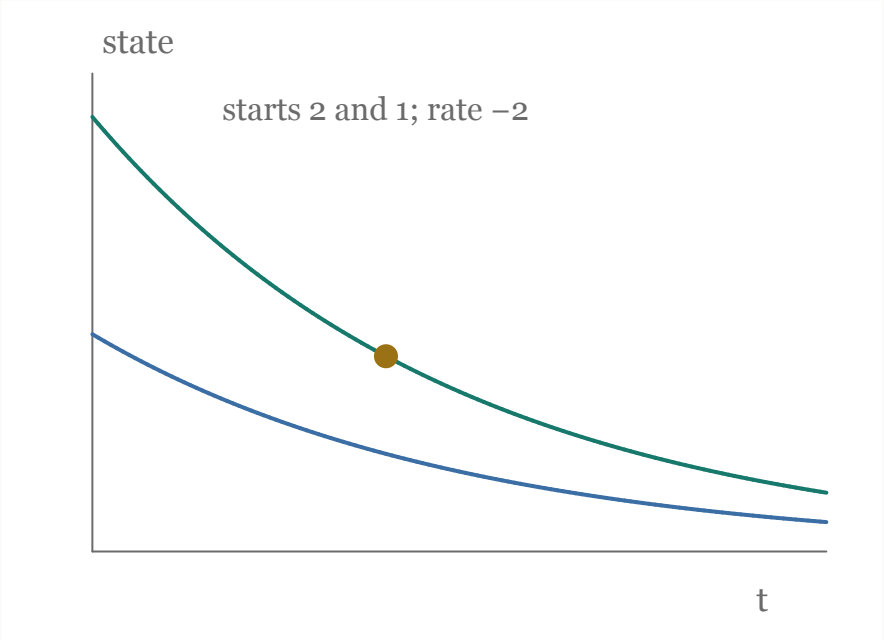
Velocity Fields

One field, different initial states

$$\dot{x} = f(x, t), \quad x(t_0) = x_0$$

- x_0 chosen start
- f fixed model
- $x(t)$ trajectory determined by the IVP

An initial-value problem combines the rule with its starting state.



WORKED EXAMPLE

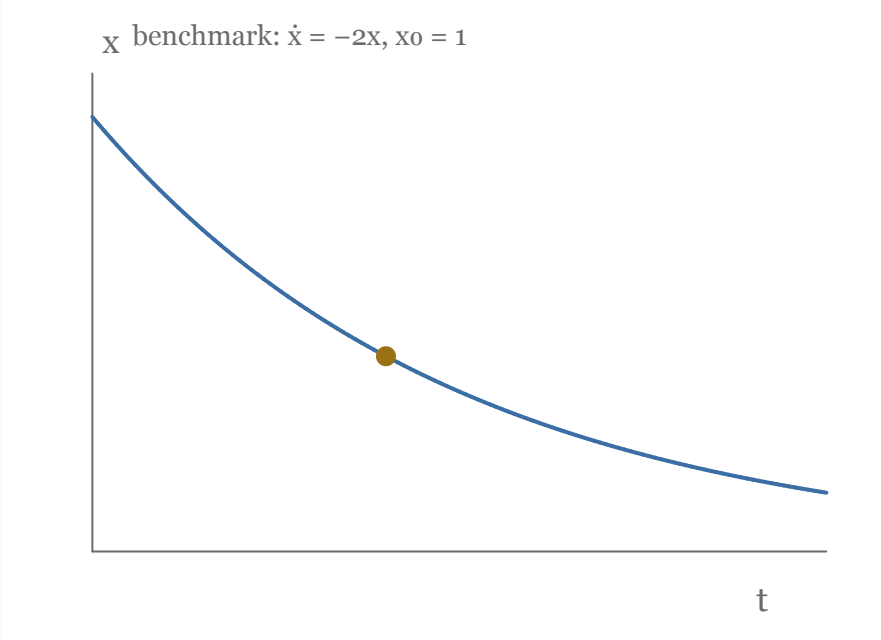
Velocity Fields

Verify an exact decay solution

$$x(t) = e^{-2t} \quad \Longrightarrow \quad \dot{x} = -2x$$

- `x(0)` one
- `-2` decay rate in inverse time
- `t` elapsed time

> Worked example: x(0.1) = 0.8187307531.



YOUR TURN

Velocity Fields

Your turn: a changed decay rate

$$\dot{x} = -3x, \qquad x(0) = 2$$

- `x` scalar state
- `-3` fixed decay rate
- `2` initial value

Find x(t), then check the differential equation and the start.

What function satisfies both conditions?

Known state → model → method



Write your rate and check its units.

The worked result appears after your attempt.

WHY

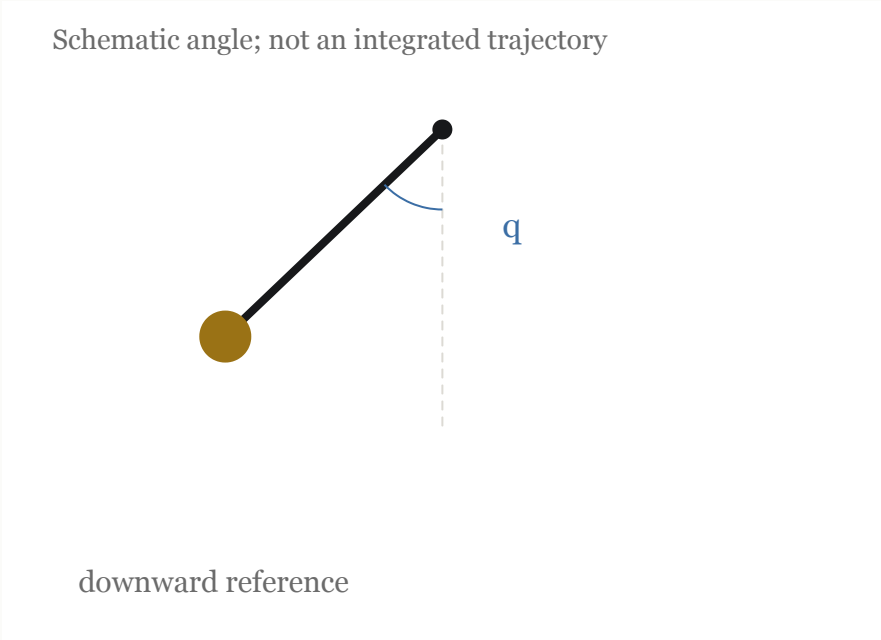
Robot State

Position alone does not predict motion

$$x = \begin{bmatrix} q \\ v \end{bmatrix}, \qquad v = \dot{q}$$

- q n joint positions
- v n joint velocities
- x 2n-component state

The same angle can occur while a joint moves in either direction.



MODEL

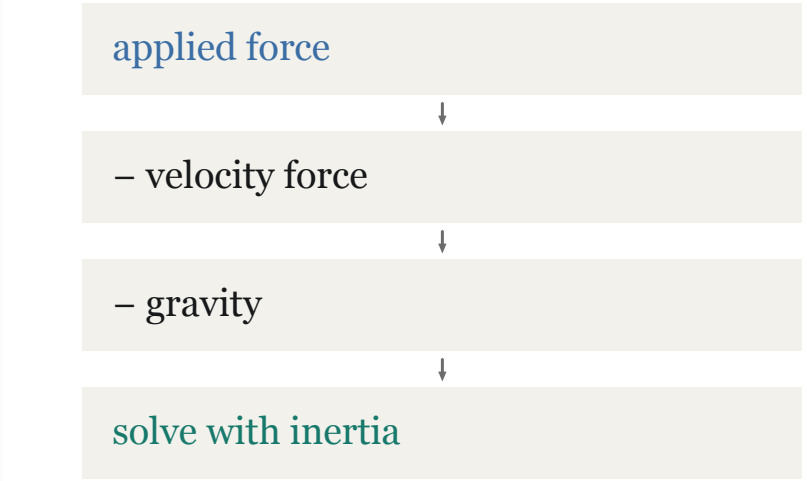
Robot State

The manipulator equation is a force balance

$$M(q)\ddot{q} + C(q, v)v + g(q) = \tau$$

`M`	n×n inertia, positive definite
`Cv`	velocity-dependent generalized forces
`g, τ`	gravity and applied generalized forces
`q,v`	positions and velocities

The model balances inertial, velocity-dependent and gravitational terms.



DERIVATION

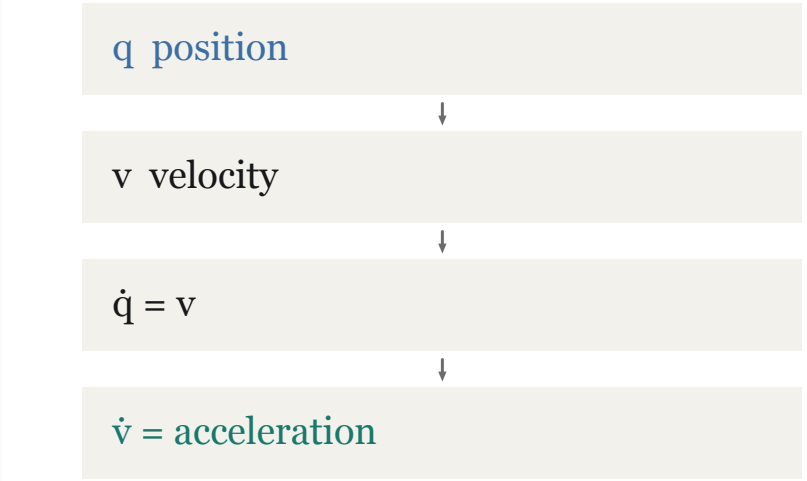
Robot State

Rewrite the mechanics as a first-order ODE

$$\dot{x} = \begin{bmatrix} v \\ M(q)^{-1}[\tau - C(q,v)v - g(q)] \end{bmatrix}$$

- x [q;v], a 2n-column
- τ applied generalized force
- M^{-1} exists for positive-definite inertia

Compute acceleration by solving M a = τ − Cv − g.



WORKED EXAMPLE

Robot State

A one-joint calculation

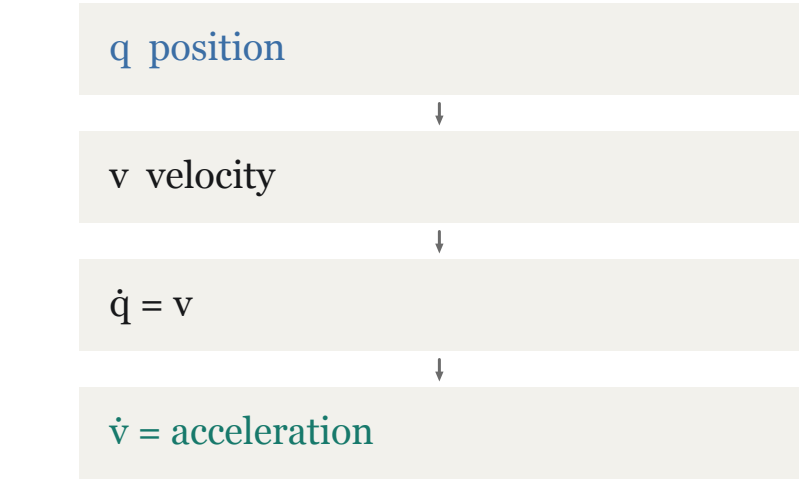
$$a = \frac{5 - 0.5(2) - 3}{2} = 0.5, \qquad \dot{x} = \begin{bmatrix} 2 \\ 0.5 \end{bmatrix}$$

`M,C` 2 and 0.5 at this state

`g,τ` 3 and 5

`q,v` 1 and 2

> Worked example: solve acceleration, then assemble the full derivative.



YOUR TURN

Robot State

Your turn: reduce the applied force

$$M = 2,\ C = 0.5,\ g = 3,\ \tau = 1,\ (q, v) = (1, 2)$$

`M,C,g` fixed at the current state

`τ` changed applied force

`q,v` unchanged state

Find both components of the state derivative.

What is the full derivative at this state?

Known state → model → method



Write your rate and check its units.

The worked result appears after your attempt.

MODEL

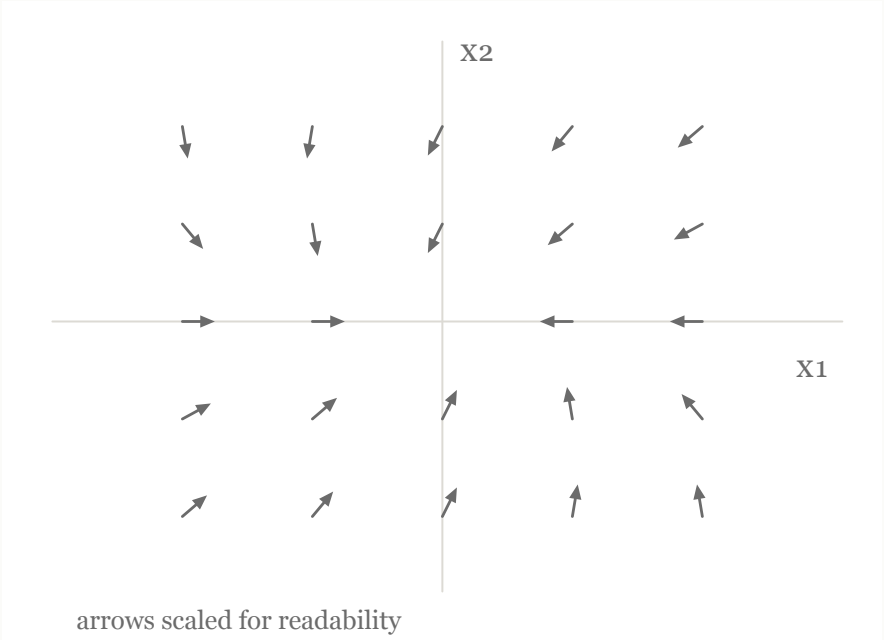
Linear Dynamics

A constant matrix gives a linear field

$$\dot{x} = Ax, \qquad A \in \mathbb{R}^{d \times d}$$

- **A** constant matrix
- **x** d-component state
- **Ax** state rate at x

No constant forcing term is hidden in this homogeneous model.



RECALL

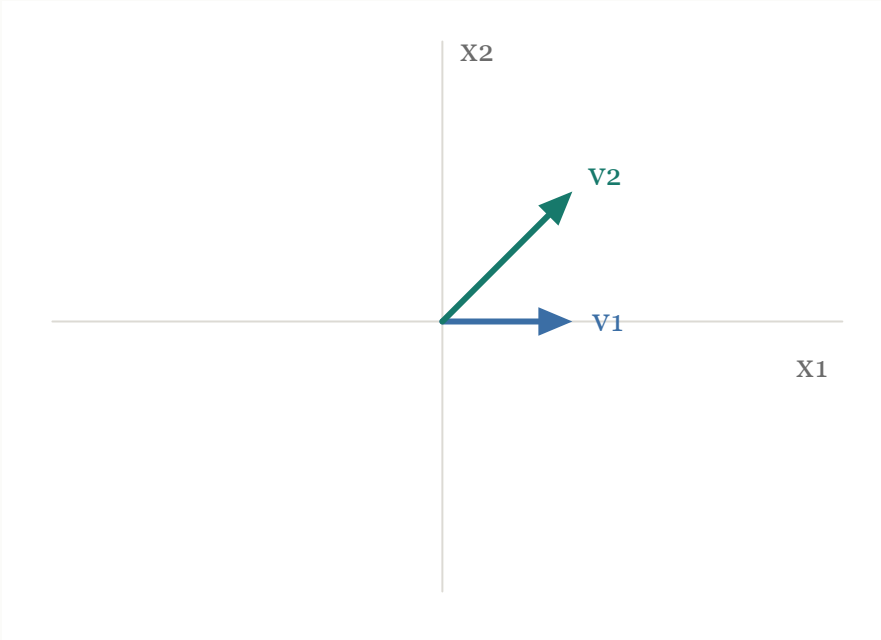
Linear Dynamics

Recall the eigenvector coordinates

$$AV = V\Lambda, \qquad x = Vc$$

- V invertible eigenvector matrix
- Λ eigenvalues in matching order
- c coordinates of x in that basis

Lecture 2: a complete eigenbasis makes the modes independent.



DERIVATION

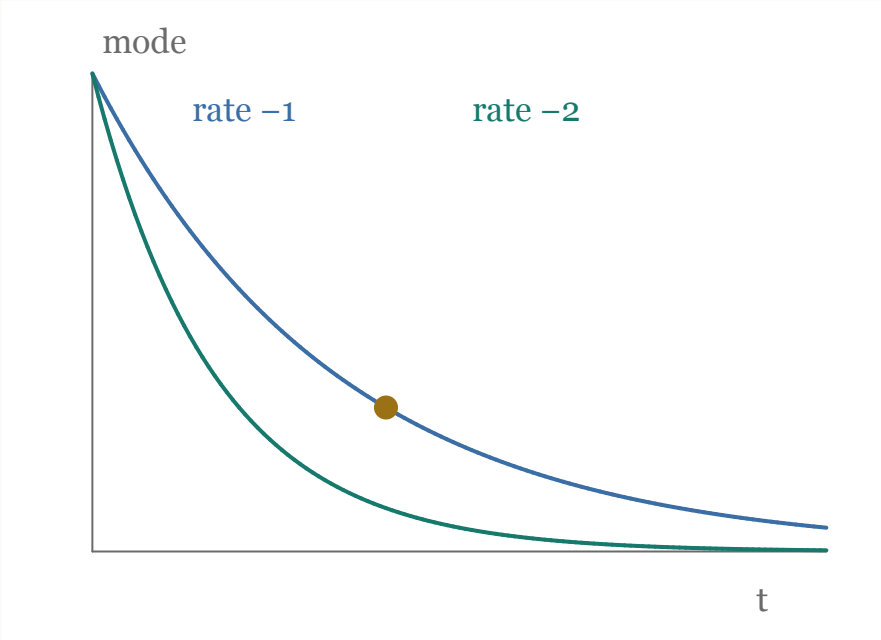
Linear Dynamics

The ODE separates into scalar modes

$$V\dot{c} = AVc = V\Lambda c \quad \Rightarrow \quad \dot{c} = \Lambda c$$

- V constant invertible basis
- c time-dependent coordinates
- Λ diagonal rate matrix

Multiply on the left by V^{-1}; each coordinate now evolves alone.



RESULT

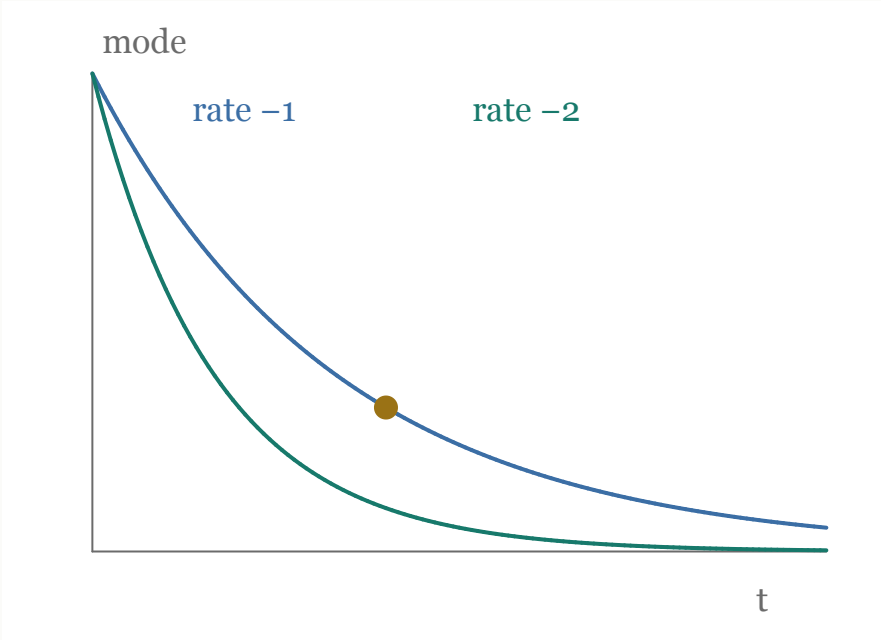
Linear Dynamics

Each eigenvalue is a continuous growth rate

$$c_i(t) = e^{\lambda_i t} c_i(0)$$

- λ_i rate of mode i
- $c_i(0)$ initial amount of that mode
- t time measured from zero

Negative rates decay; positive rates grow if the mode is present.



YOUR TURN

Linear Dynamics

Return from modal coordinates

$$x(t) = V e^{\Lambda t} V^{-1} x_0 = \begin{bmatrix} e^{-t} + e^{-2t} \\ e^{-2t} \end{bmatrix}$$

- V [[1,1],[0,1]]
- Λ diag(−1,−2)
- $V^{-1}x_0$ (1,1): initial modal coordinates

> A=[[-1,−1],[0,−2]], x0=(2,1).

Change x0 to (1,1). Find its modal coordinates and trajectory.

Known state → model → method



Write your rate and check its units.

The worked result appears after your attempt.

DEFINITION

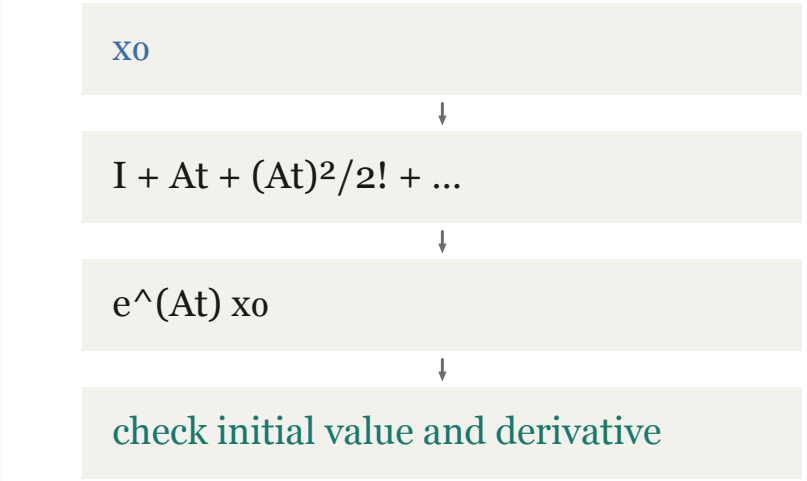
Linear Dynamics

The matrix exponential always exists

$$e^{At} = \sum_{j=0}^{\infty} \frac{(At)^j}{j!}, \qquad x(t) = e^{At} x_0$$

- A any square constant matrix
- j series index; $A^0 = I$
- x_0 initial state

Diagonalization is one way to evaluate the solution, not an existence requirement.



YOUR TURN

Linear Dynamics

A defective matrix still has a solution

$$A = \begin{bmatrix} -1 & 1 \\ 0 & -1 \end{bmatrix}, \quad x_0 = (0, 1)^T$$

- `A` one eigenvector direction
- `x0` known initial state
- `t` elapsed time

Verify $x(t) = (t\,e^{-t}, e^{-t})^T$.

Does the proposed trajectory satisfy the IVP?

Known state → model → method



Write your rate and check its units.

The worked result appears after your attempt.

WORKED EXAMPLE

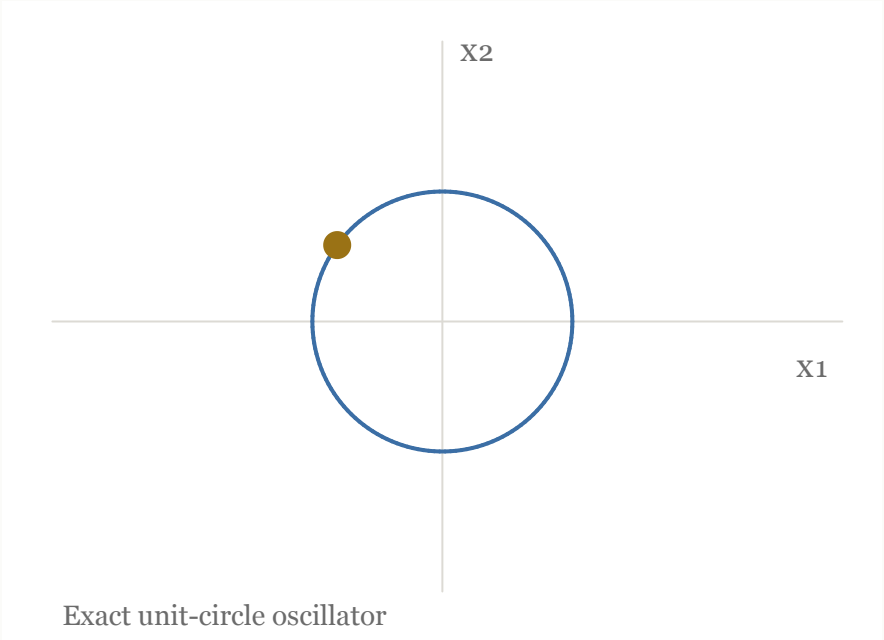
Linear Dynamics

A real rotation has complex eigenvalues

$$A = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}, \quad x(t) = (\cos t, \sin t)^T$$

- x_0 $(1,0)^T$
- A constant rotation generator
- λ $\pm i$ over the complex field

> This trajectory stays on the unit circle.



Exact unit-circle oscillator

YOUR TURN

Linear Dynamics

Zero real parts need a closer look

$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \qquad x_0 = (0, 1)^T$$

- `A` both eigenvalues zero
- `x0` chosen initial state
- `x(t)` state to determine

Do zero eigenvalues guarantee a bounded trajectory?

What happens from the given start?

Known state → model → method



Write your rate and check its units.

The worked result appears after your attempt.

DERIVATION

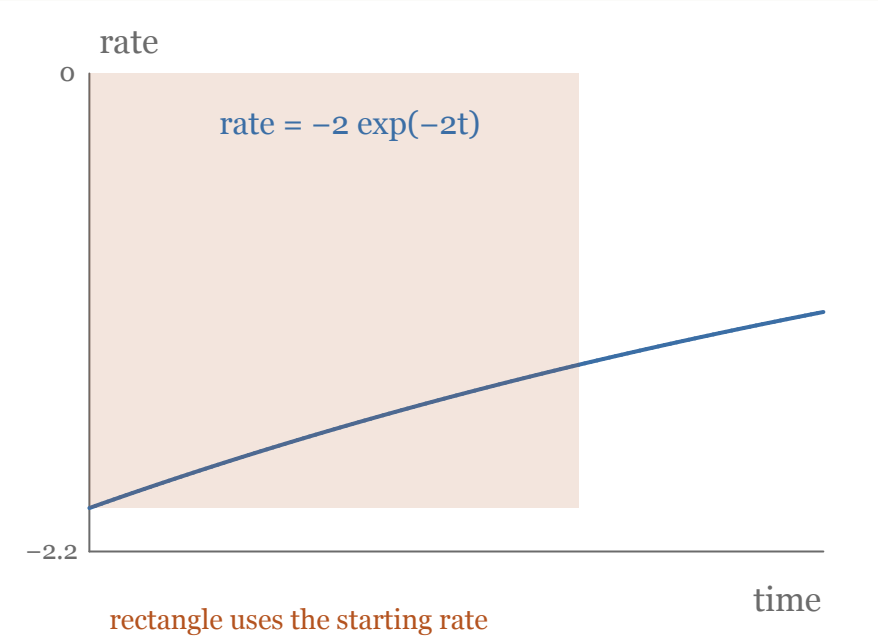
Euler Integration

An exact step is an integral

$$x(t+h) - x(t) = \int_t^{t+h} f(x(s), s)\, ds$$

- `h` positive interval length
- `s` integration variable
- `f(x(s),s)` rate along the unknown path

The fundamental theorem of calculus gives the starting relation.



RULE

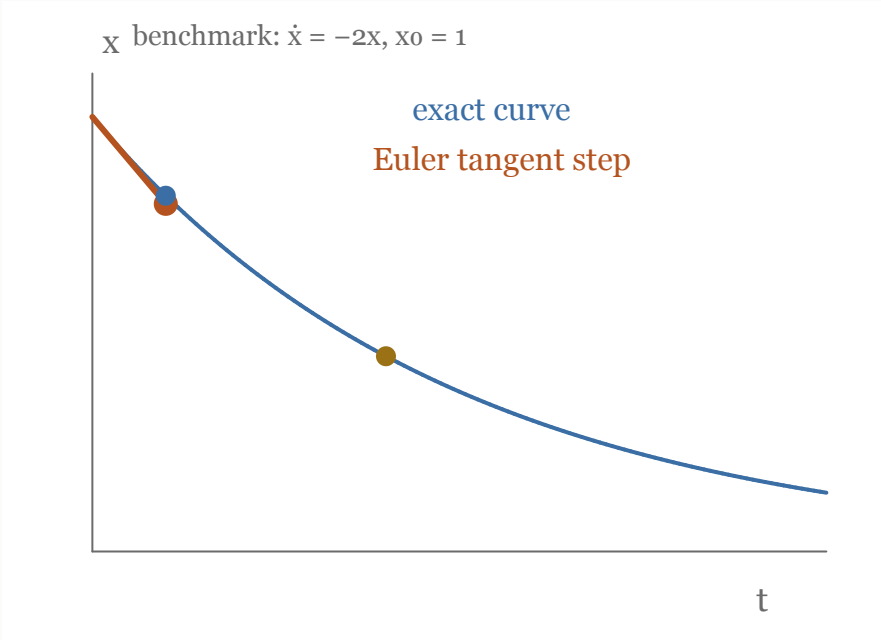
Euler Integration

Euler uses the rate at the start

$$x_{k+1} = x_k + hf(x_k, t_k)$$

- x_k current numerical state
- h positive time step
- $f(x_k, t_k)$ rate at the old state

Every component uses the same old state.



WORKED EXAMPLE

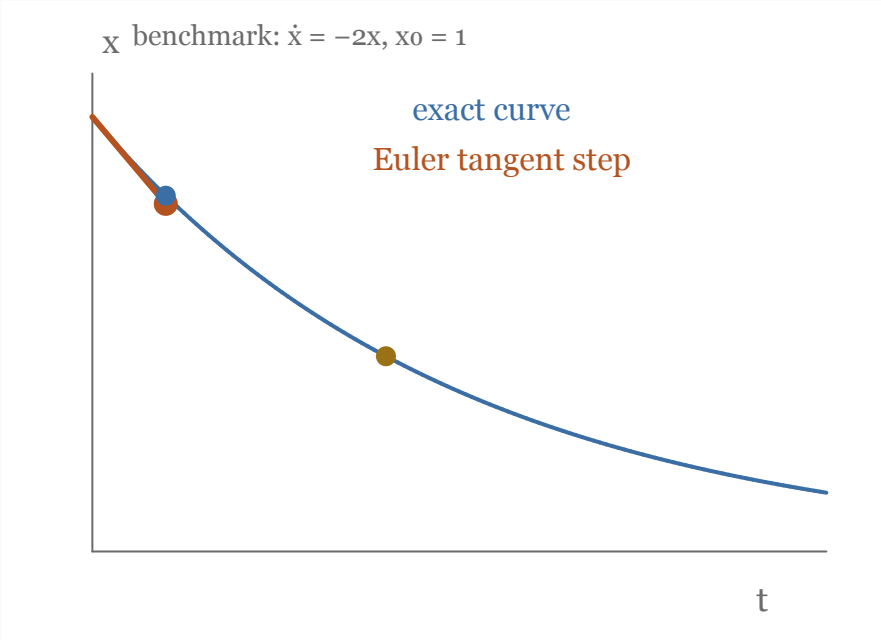
Euler Integration

One step against an exact answer

$$x_1 = 1 + 0.1(-2) = 0.8$$

x_0	one
h	0.1
$f(x)$	$-2x$

> Worked example: exact x(0.1)=0.8187307531; Euler is lower.



YOUR TURN

Euler Integration

Your turn: two larger steps

$$\dot{x} = -2x, \quad x_0 = 1, \quad h = 0.2$$

- `h` fixed time step
- x_0 initial numerical state
- x_2 approximation at t=0.4

Compute x1 and x2. Compare x2 with the exact state at the same time.

Which states result after one and two steps?

Known state → model → method



Write your rate and check its units.

The worked result appears after your attempt.

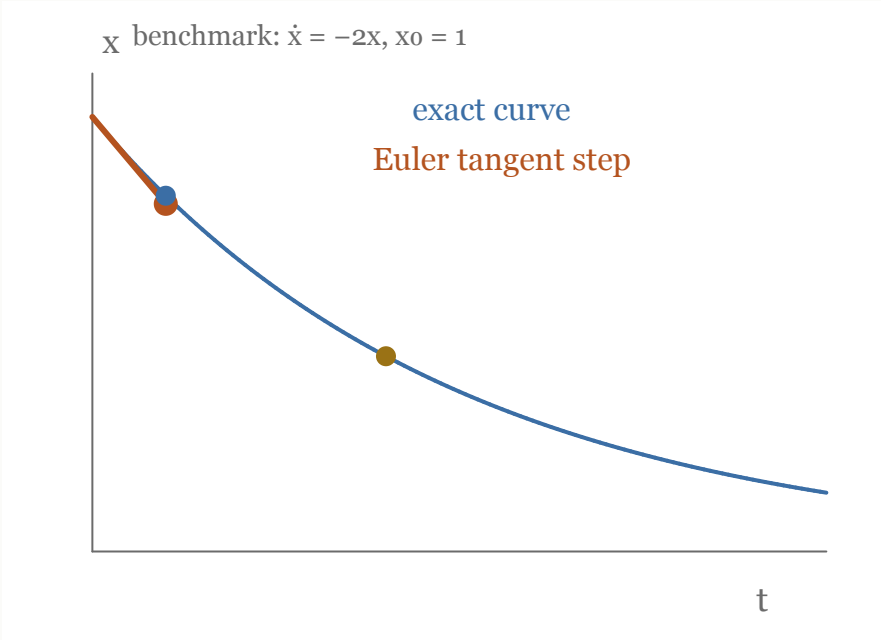
Euler Integration

Local error and accumulated error differ

$$x(t+h) = x(t) + h\dot{x}(t) + \tfrac{1}{2}h^2\ddot{x}(t) + O(h^3)$$

- x sufficiently smooth exact solution
- h small step
- $\ddot{x}$ change of the velocity along the path

Assume f has continuous derivatives through order two near the finite trajectory; bound them there.



RECALL AND DERIVATION

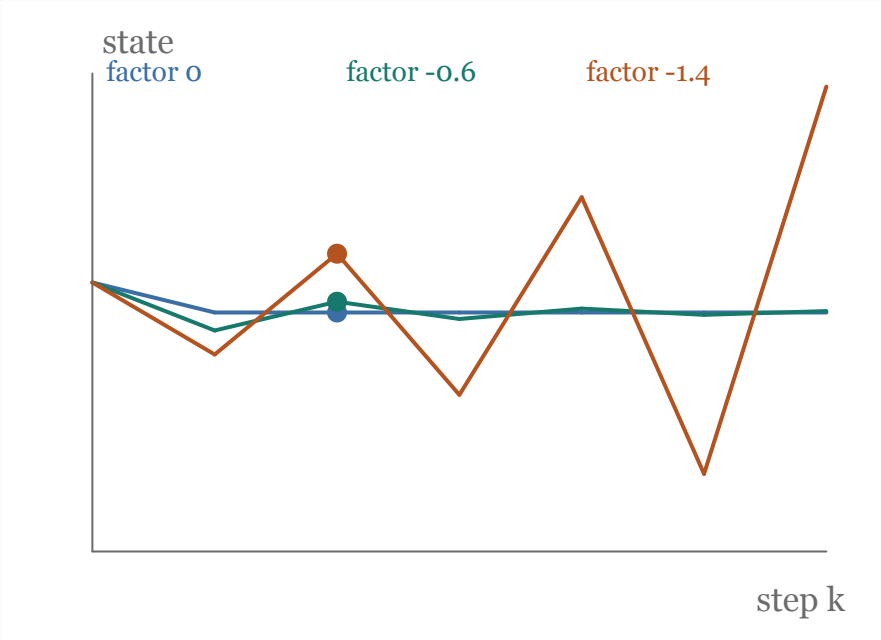
Numerical Stability

A repeated step has a multiplier

$$\dot{x} = \lambda x \quad \Rightarrow \quad x_{k+1} = (1 + h\lambda)x_k$$

- λ constant scalar rate
- h step size
- $1+h\lambda$ numerical amplification factor

Recall Lecture 6: A=−H turns this into the gradient-descent multiplier.



DERIVATION

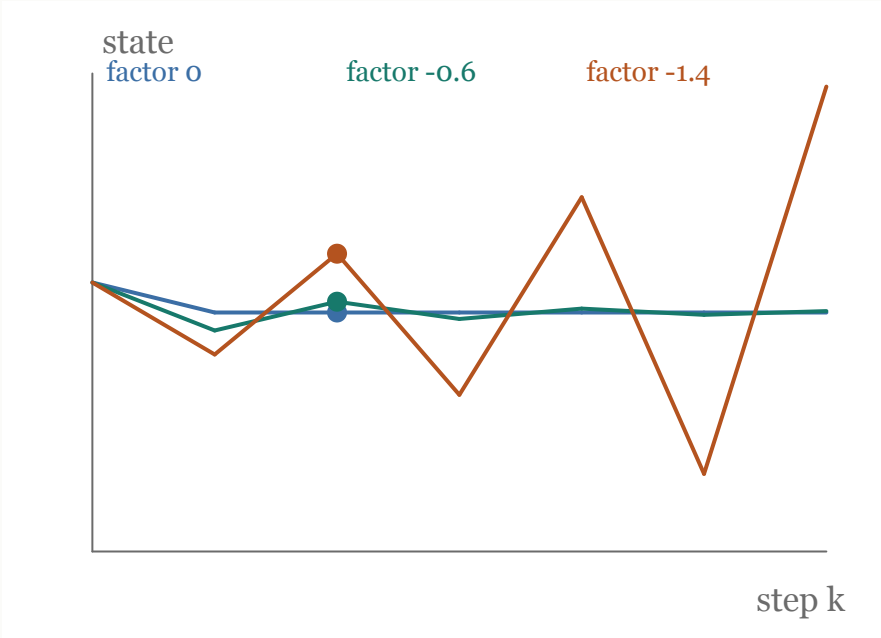
Numerical Stability

The strict inequality decides decay

$$|1 + h\lambda| < 1 \quad \Longleftrightarrow \quad 0 < h < -2/\lambda$$

- λ real negative rate
- h positive step size
- $|1+h\lambda|$ size of each numerical multiplier

At the endpoint the factor is -1: oscillation without decay.



YOUR TURN

Numerical Stability

Predict three step-size regimes

$$\lambda = -4, \qquad h \in \{0.25, 0.4, 0.6\}$$

- λ fixed decay rate
- h candidate steps
- x_0 any nonzero start

Classify each Euler sequence: zero, alternating decay, or growth.

What does each numerical trajectory do?

Known state → model → method



Write your rate and check its units.

The worked result appears after your attempt.

YOUR TURN

Numerical Stability

Every mode must satisfy the bound

$$A = \operatorname{diag}(-2, -8)$$

- A constant state matrix
- h one step used for both modes
- x_0 arbitrary initial state

Require decay from every start.

Which condition works from every initial state?

Known state → model → method



Write your rate and check its units.

The worked result appears after your attempt.

DIAGNOSTIC

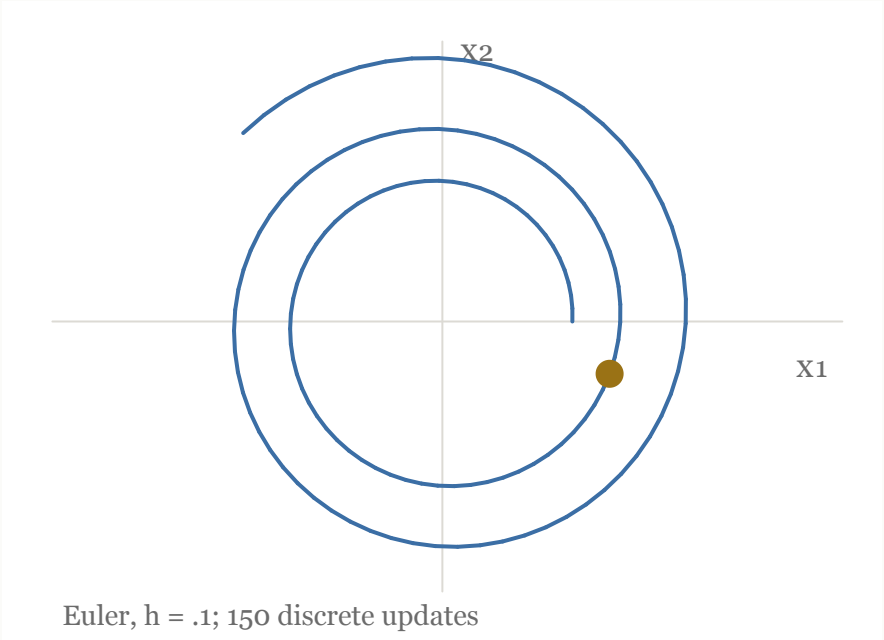
Numerical Stability

Euler adds energy to an oscillator

$$|1 + ih| = \sqrt{1 + h^2} > 1 \quad (h > 0)$$

- `±i` exact oscillator eigenvalues
- `h` positive Euler step
- `|1+ih|` numerical amplification magnitude

A bounded exact orbit can produce an outward numerical spiral.



Euler, h = .1; 150 discrete updates

RECAP

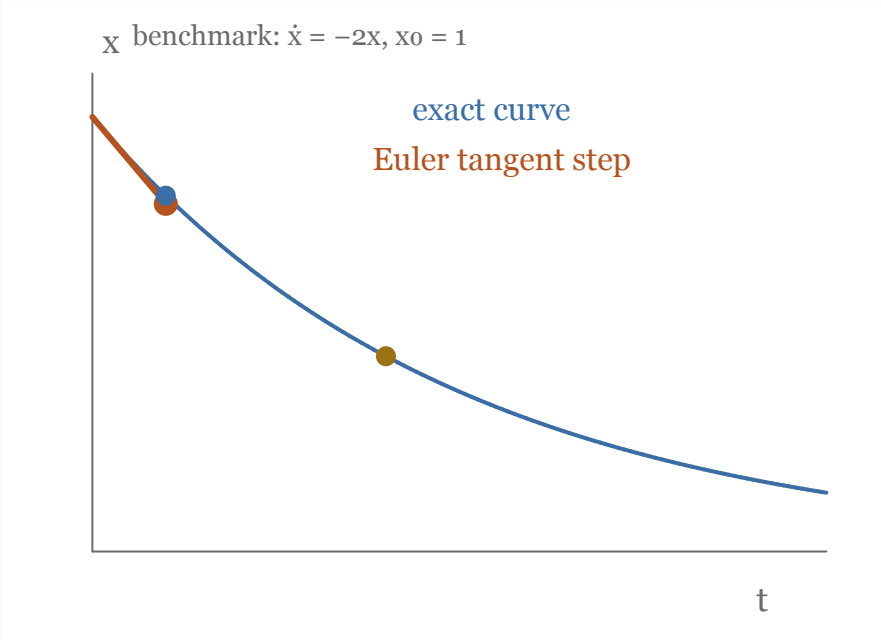
Numerical Stability

What did the numerical step change?

$$\text{model: } e^{\lambda h} \qquad \text{Euler: } 1 + h\lambda$$

- `λ` continuous rate
- `h` step size
- `two factors` exact and numerical evolution over the same interval

A correct model and a correct update formula can still give a poor simulation.



WHY

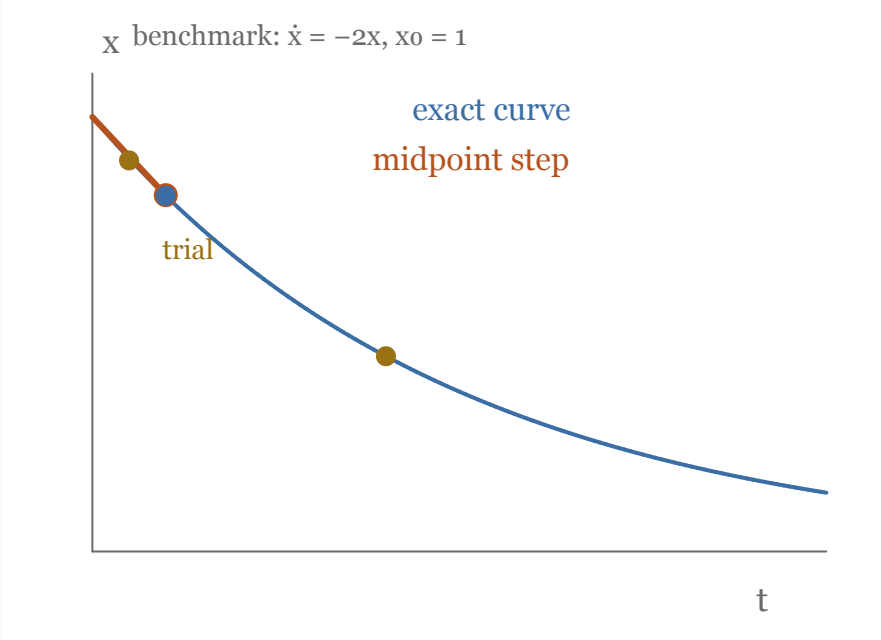
Runge–Kutta Methods

Sample the field inside the step

$$\Delta x = \int_{t_k}^{t_k+h} f(x(s), s)\, ds$$

- Δx exact increment from the current exact state
- f same velocity model
- h one integration interval

A better slope estimate can improve the same time step.



RULE

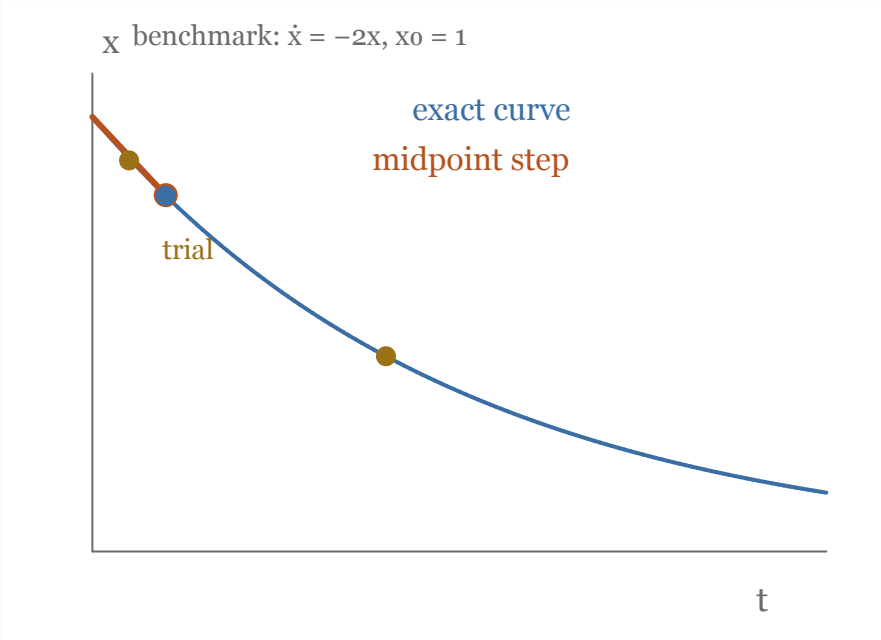
Runge–Kutta Methods

Construct a trial midpoint

$$k_1 = f(x_k, t_k), \qquad k_2 = f(x_k + \tfrac{h}{2}k_1, t_k + \tfrac{h}{2})$$

k_1, k_2	rates, not increments
$h\ k_1/2$	trial half-step displacement
$t_k + h/2$	midpoint time

The first rate predicts a location; the second samples the field there.



WORKED EXAMPLE

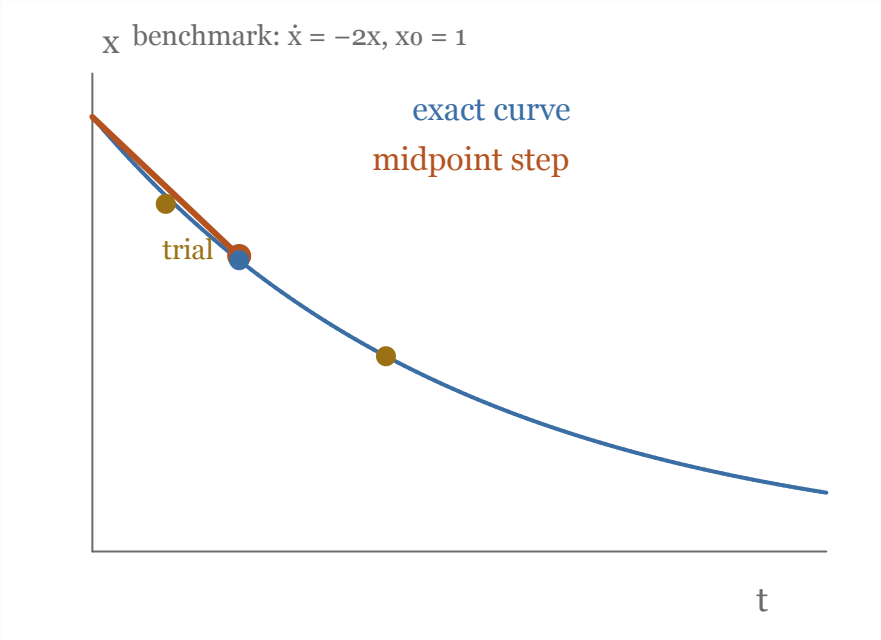
Runge–Kutta Methods

Advance with the midpoint rate

$$x_{k+1} = x_k + hk_2$$

- x_k original current state
- k_2 rate at the trial midpoint
- h full step length

> $\dot{x}=-2x$, $x_0=1$, $h=.2$: $k_1=-2 \rightarrow$ trial $=.8 \rightarrow k_2=-1.6 \rightarrow x_1=.68$.



YOUR TURN

Runge–Kutta Methods

Your turn: halve the midpoint step

$$\dot{x} = -2x, \quad x_0 = 1, \quad h = 0.1$$

- k_1, k_2 two model evaluations
- h full step
- x_1 accepted next state

Write both stages and the accepted state.

What is the accepted midpoint result?

Known state → model → method



Write your rate and check its units.

The worked result appears after your attempt.

RULE

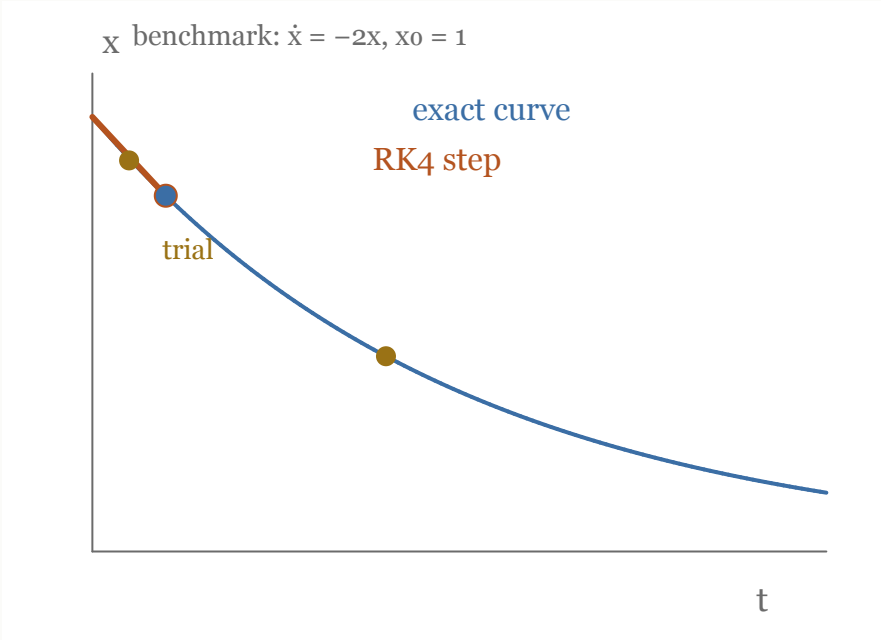
Runge–Kutta Methods

Classical RK4 evaluates four trial rates

$$
\begin{aligned}
k_1 &= f(x, t) \\
k_2 &= f(x + hk_1/2, t + h/2) \\
k_3 &= f(x + hk_2/2, t + h/2) \\
k_4 &= f(x + hk_3, t + h)
\end{aligned}
$$

- `x,t` current state and time
- `h` one full step
- `k₁…k₄` four rates with state/time units

Each trial is anchored at the original x.



WORKED EXAMPLE

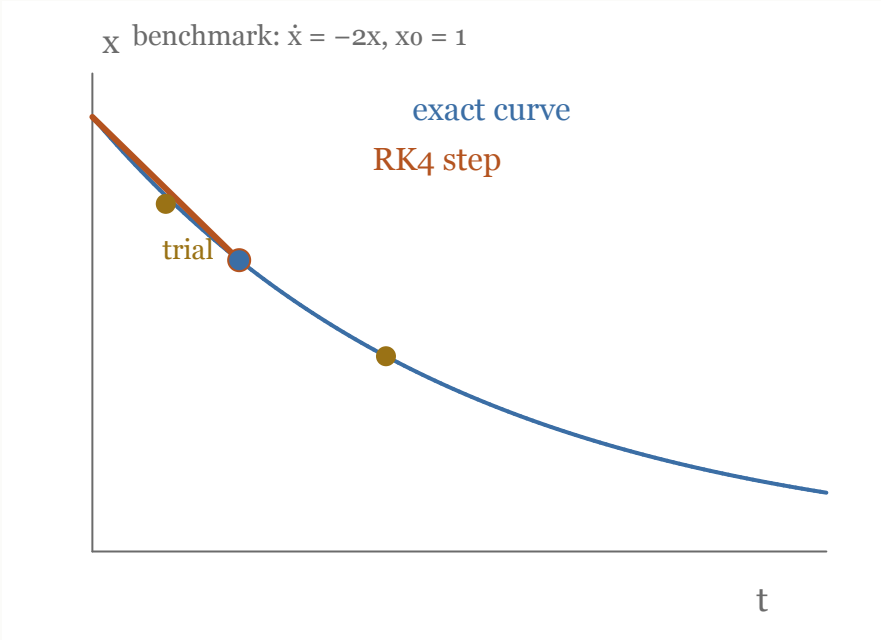
Runge–Kutta Methods

Combine the four rates

$$x_{k+1} = x_k + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

$k_1 \ldots k_4$	RK4 rates
1,2,2,1	weights before division by six
h	full step length

> $\dot{x}=-2x$, $x_0=1$, $h=.2$: rates $(-2,-1.6,-1.68,-1.328)$ give $x_1=.6704$.



YOUR TURN

Runge–Kutta Methods

Your turn: RK4 and model-call cost

$$\dot{x} = -2x, \quad x_0 = 1, \quad h = 0.1$$

`h`	full step
`k₁…k₄`	rates to compute
`cost`	number of calls to f

Compute four rates and count calls.

What does one step return, at what model-call cost?

Known state → model → method



Write your rate and check its units.

The worked result appears after your attempt.

COMPARISON

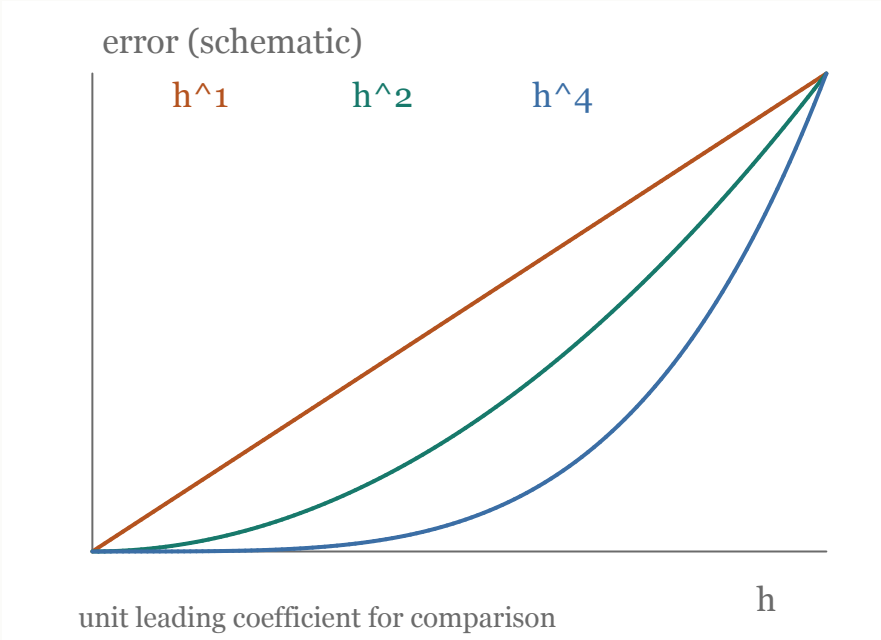
Runge–Kutta Methods

Accuracy improves at a computational price

$$E_{\text{Euler}} = O(h), \quad E_{\text{midpoint}} = O(h^2), \quad E_{\text{RK4}} = O(h^4)$$

E	fixed-final-time global error
h	step tending to zero
assumptions	sufficient smoothness and a well-posed finite interval

Halve h and check convergence; RK4 is not unconditionally stable.



MODEL

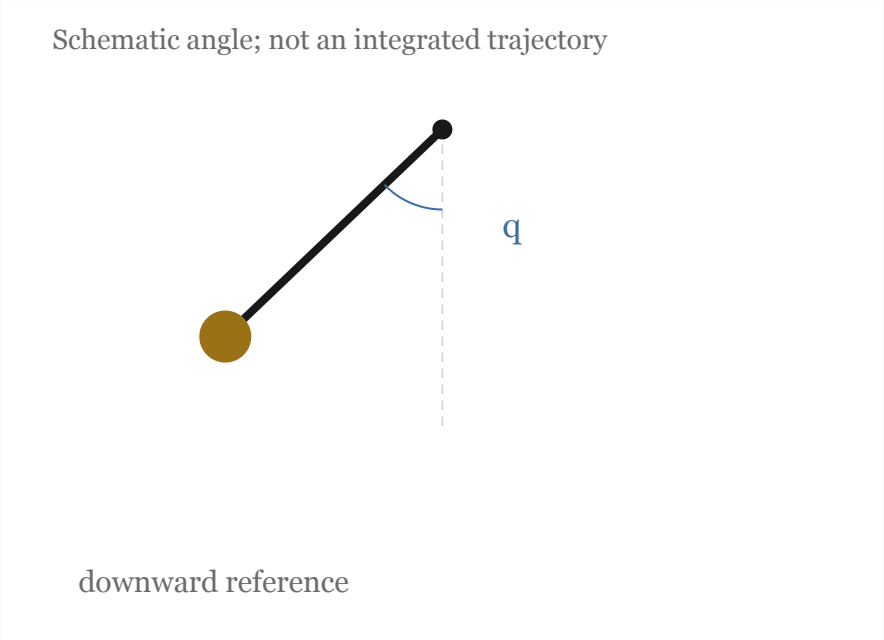
Pendulum Simulation

The restoring acceleration has a sign

$$\dot{q} = v, \qquad \dot{v} = -\frac{g_0}{\ell} \sin q$$

- `q` angle from downward vertical
- `v` angular velocity
- `g₀, ℓ` positive gravity acceleration and length

Unforced, undamped motion is the model for this comparison.



DEFINITION

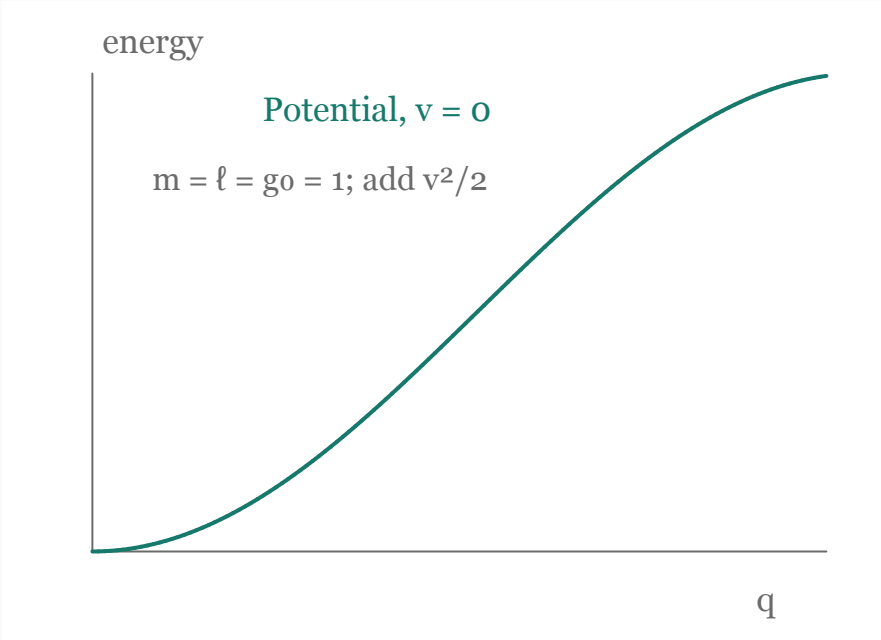
Pendulum Simulation

Mechanical energy is a diagnostic

$$E(q, v) = \tfrac{1}{2} m \ell^2 v^2 + m g_0 \ell (1 - \cos q)$$

`first term`	kinetic energy
`second term`	potential energy above the bottom
`m, ℓ, g₀`	fixed positive constants

The zero of potential energy is a convention; energy differences have meaning.



DERIVATION

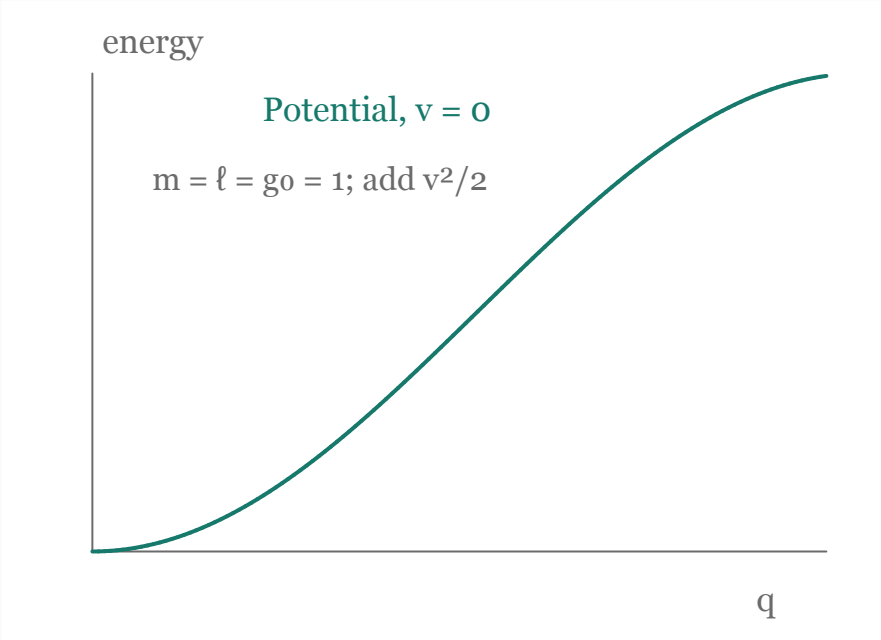
Pendulum Simulation

The exact dynamics conserve energy

$$\dot{E} = m\ell^2 v\dot{v} + mg_0\ell \sin q\, \dot{q} = 0$$

- $\dot{q}=v$ kinematic identity
- $\dot{v}$ $-(g_0/\ell)\sin q$
- $\dot{E}$ rate of mechanical energy

The two terms cancel only for the stated unforced, undamped model.



INTEGRATED BOARD

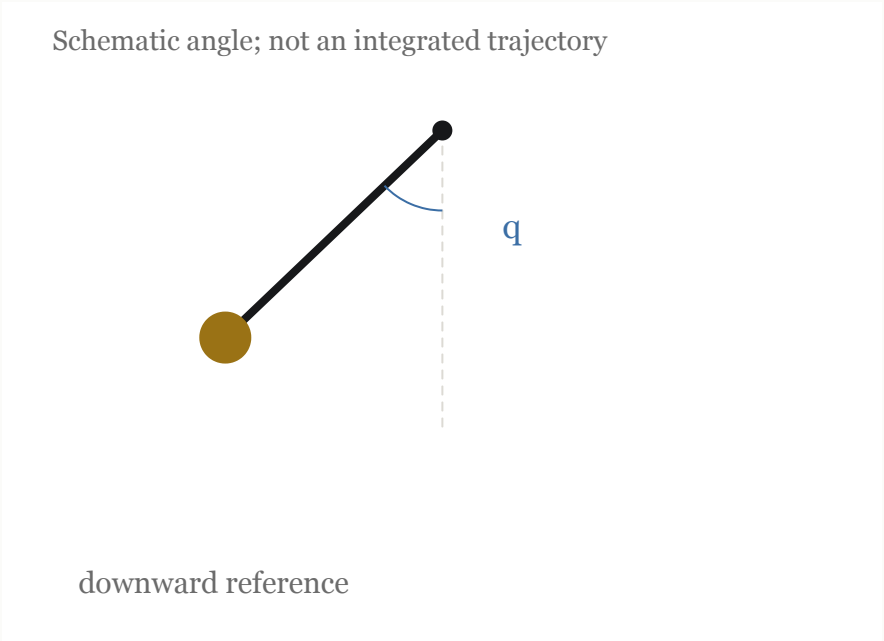
Integrated Pendulum Problem

Choose the state and the check first

$$m = \ell = g_0 = 1, \quad x_0 = (1, 0)^T, \quad h = 0.1$$

- `q0` one radian
- `v0` zero
- `h` one time step
- `E` v²/2+1−cos q

Find one numerical step, then test its energy.



WORKED EXAMPLE

Integrated Pendulum Problem

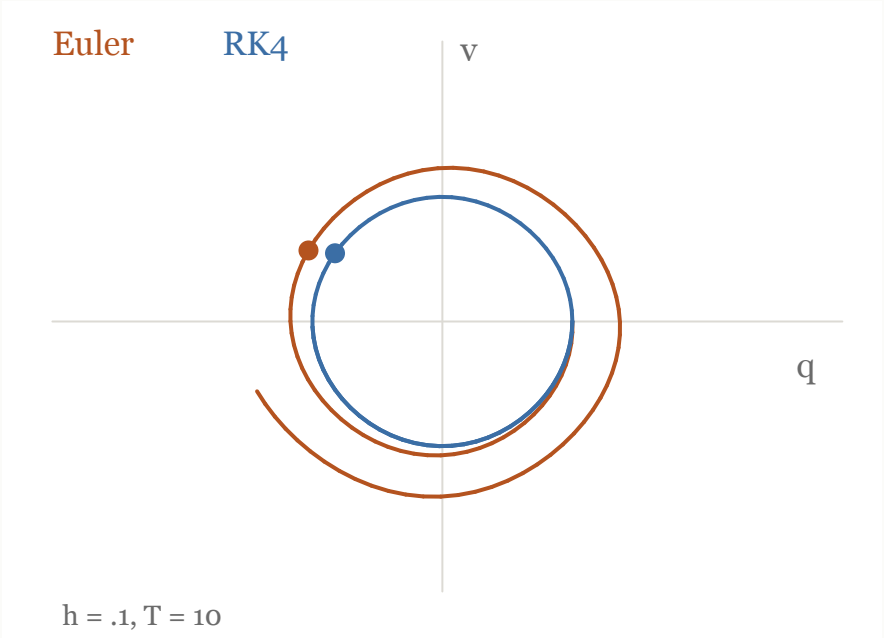
Compare the first accepted states

method	q_1	v_1
Euler	1	−0.08414710
midpoint	0.99579265	−0.08414710
RK4	0.99579454	−0.08407114

`q₁,v₁` accepted state after h=.1

`all methods` same model and initial state

> Euler leaves q unchanged on its first step because v0=0.



EXPERIMENT

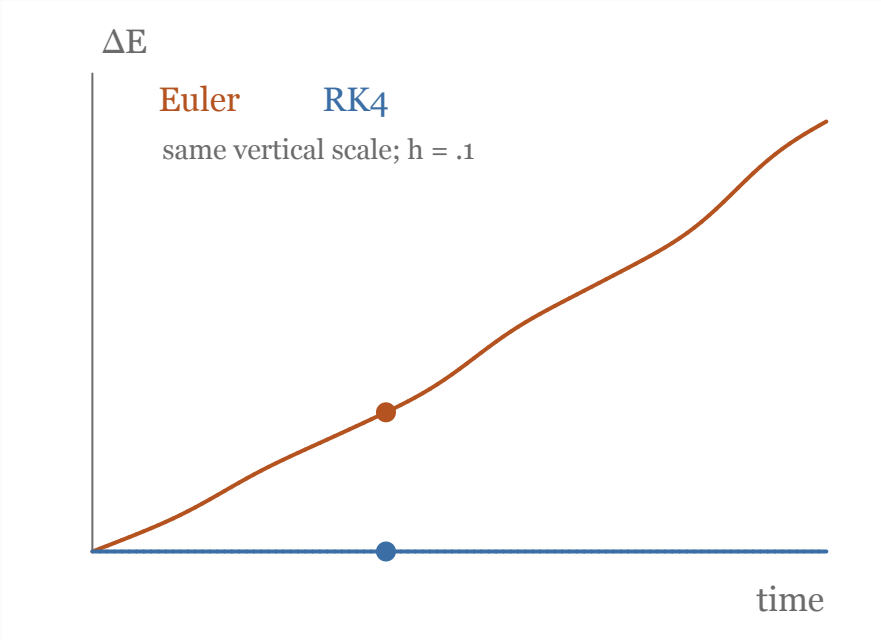
Integrated Pendulum Problem

Repeat to the same final time

h	ΔE_{Euler}	ΔE_{RK4}
.2	1.324213	-1.46×10^{-5}
.1	.539746	-4.61×10^{-7}
.05	.232375	-1.44×10^{-8}

- ΔE E(xN)−E(xo)
- T 10, fixed for every run
- N T/h steps

Same initial state, same dynamics, same endpoint; only method and h change.



INTERPRETATION

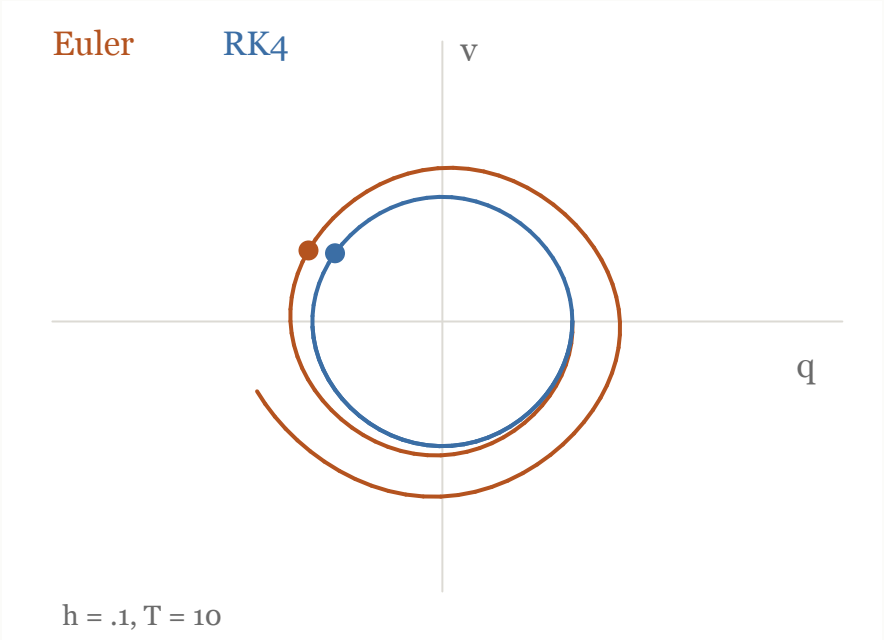
Integrated Pendulum Problem

Energy error does not measure phase error

$$E(x_k) \approx E(x_0) \not\Rightarrow x_k \approx x(t_k)$$

- E one scalar diagnostic
- x_k two-component numerical state
- $x(t_k)$ exact state at the same time

Compare angle, velocity and energy; then halve the step.



YOUR TURN

Independent Transfer

A different angle, the same method choice

$$q_0 = 0.5, \quad v_0 = 0, \quad h = 0.1, \quad m = \ell = g_0 = 1$$

- q_0 new angle in radians
- v_0 zero angular velocity
- h positive time step

One model call. Choose a method, then compute the step.

Is energy conserved?

Known state → model → method



Write your rate and check its units.

The worked result appears after your attempt.

YOUR TURN

Simulation Checklist

Cold-write a complete simulation step

$$\dot{x} = f(x, t), \qquad x(t_0) = x_0$$

- `x` complete state
- `f` model rate
- `h` chosen integration step

Name the state, update, stability question and diagnostic.

What four checks make the result interpretable?

Known state → model → method



Write your rate and check its units.

The worked result appears after your attempt.

CONNECTION

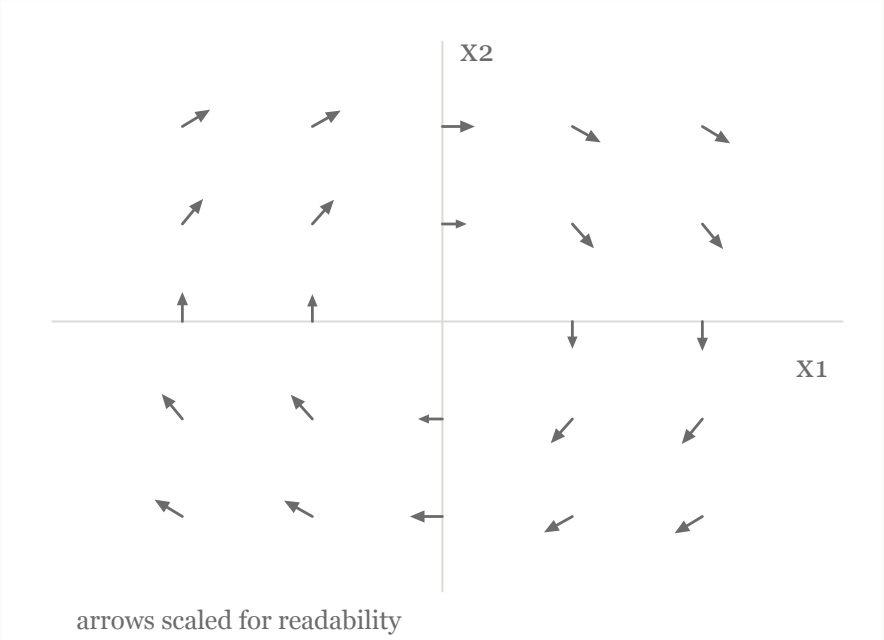
Simulation Checklist

A learned field still needs an integrator

$$\dot{x} = v_\theta(x, t)$$

- `vθ` a field represented by a learned model
- `x` generated state
- `t` continuous evolution coordinate

Later: flow matching supplies a field; Euler or RK advances it.



References

Sources for models and exact dynamics

1. Tedrake, R. Underactuated Robotics, ch.2. MIT course notes, 2024.

2. Lynch, K. M. & Park, F. C. Modern Robotics, ch.8. Cambridge, 2017.

3. Strang, G. Differential Equations and exp(At). MIT 18.06 Lecture 23, 2010.

References

Sources for integration and stability

1. Kværnø, A. Numerical Solution of ODEs, part 2. NTNU, 2022.

2. Driscoll, T. & Braun, R. Fundamentals of Numerical Computation, §6.4. Online textbook.